

Study Notes Series

2026 Spring

吴恩达机器学习笔记 (3)

课程：机器学习 作者：C++ 与草稿纸

本节摘要

本节围绕分类问题与过拟合问题，介绍逻辑回归从模型表示、代价函数到参数优化的完整流程，并说明如何使用正则化提升模型对新数据的泛化能力。

- **逻辑回归**：使用 Sigmoid 函数把线性组合映射为 0 到 1 之间的概率；
- **判定边界**：由参数决定输入空间中不同预测类别的分界，可通过多项式特征形成非线性边界；
- **模型训练**：使用二元交叉熵代价函数，并通过梯度下降或高级优化算法求解参数；
- **多类别分类**：训练多个“一对多”二元分类器，选择预测概率最大的类别；
- **正则化**：在代价函数中惩罚过大的参数，缓解过拟合，同时通过参数 λ 控制拟合与简化模型之间的平衡。

1 分类与逻辑回归

1.1 分类问题

二元分类

分类问题的目标是预测离散类别，而不是连续数值。对于二元分类，通常把两个类别分别记为

$$y \in \{0, 1\},$$

其中 0 表示负类 (negative class)，1 表示正类 (positive class)。例如，判断邮件是否为垃圾邮件、交易是否存在欺诈、肿瘤是良性还是恶性，都可以写成二元分类问题。

为什么不直接使用线性回归分类

线性回归的输出没有范围限制，可能小于 0 或大于 1。即使人为规定“预测值不小于 0.5 时判为正类”，新加入的极端样本也可能明显改变拟合直线和分类结果。因此，线性回归的输出既不适合解释为概率，也通常不是分类任务的合理模型。逻辑回归虽然名称中含有“回归”，但它本质上是一个分类算法。

1.2 Sigmoid 函数与假设表示

Sigmoid 函数

逻辑回归使用 Sigmoid 函数，也称 Logistic 函数：

$$g(z) = \frac{1}{1 + e^{-z}}.$$

它把任意实数映射到区间 $(0, 1)$ ，并具有以下性质：

$$g(0) = 0.5, \quad z > 0 \Rightarrow g(z) > 0.5, \quad z < 0 \Rightarrow g(z) < 0.5.$$

当 z 趋向正无穷时， $g(z)$ 趋近于 1；当 z 趋向负无穷时， $g(z)$ 趋近于 0。

逻辑回归模型

令 $x_0 = 1$ ，特征向量和参数向量分别为

$$\mathbf{x} = [x_0, x_1, \dots, x_n]^T, \quad \boldsymbol{\theta} = [\theta_0, \theta_1, \dots, \theta_n]^T.$$

逻辑回归模型为

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = g(\boldsymbol{\theta}^T \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}}.$$

模型输出可以解释为在给定输入和参数时属于正类的估计概率：

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}).$$

因此

$$P(y = 0 \mid \mathbf{x}; \boldsymbol{\theta}) = 1 - h_{\boldsymbol{\theta}}(\mathbf{x}).$$

例如，若 $h_{\boldsymbol{\theta}}(\mathbf{x}) = 0.7$ ，则模型认为该样本属于正类的概率为 70%。

概率与最终类别

模型输出的是概率，最终类别还需要通过阈值获得。默认阈值通常取 0.5：

$$\hat{y} = \begin{cases} 1, & h_{\theta}(\mathbf{x}) \geq 0.5, \\ 0, & h_{\theta}(\mathbf{x}) < 0.5. \end{cases}$$

实际应用中可以根据漏判与误判的代价调整阈值；改变阈值会改变分类结果，但不会重新训练模型参数。

1.3 判定边界

判定边界

当阈值为 0.5 时，Sigmoid 函数的单调性给出

$$h_{\theta}(\mathbf{x}) \geq 0.5 \iff \boldsymbol{\theta}^T \mathbf{x} \geq 0.$$

所以方程

$$\boldsymbol{\theta}^T \mathbf{x} = 0$$

描述的集合就是**判定边界（Decision Boundary）**。判定边界由模型参数和特征表示共同决定，而不是训练样本本身直接构成。

线性判定边界

若只有两个特征，并且

$$\boldsymbol{\theta} = \begin{bmatrix} -3 & 1 & 1 \end{bmatrix}^T,$$

则

$$\boldsymbol{\theta}^T \mathbf{x} = -3 + x_1 + x_2.$$

判定边界为直线 $x_1 + x_2 = 3$ 。直线一侧满足 $x_1 + x_2 \geq 3$ ，预测为 1；另一侧预测为 0。

非线性判定边界

逻辑回归对参数仍然是线性的，但可以使用多项式特征表示弯曲边界。例如

$$h_{\theta}(\mathbf{x}) = g(-1 + x_1^2 + x_2^2)$$

对应的判定边界为

$$x_1^2 + x_2^2 = 1,$$

即以原点为圆心、半径为 1 的圆。加入更多高次项或交叉项可以得到更复杂的边

界，但模型越复杂，过拟合风险通常越高。

2 逻辑回归的代价函数与训练

2.1 二元交叉熵代价函数

不能直接照搬平方误差

将 Sigmoid 模型代入线性回归的平方误差代价函数，会得到不利于梯度下降求解的非凸目标。逻辑回归改用对数损失，使代价函数成为凸函数，从而避免多个局部最小值带来的优化困难。

单个样本的对数损失

令 $\hat{y} = h_{\theta}(\mathbf{x})$ ，则一个训练样本的损失为

$$L(\hat{y}, y) = \begin{cases} -\log(\hat{y}), & y = 1, \\ -\log(1 - \hat{y}), & y = 0. \end{cases}$$

两种情况可以合并为

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}).$$

当预测正确且置信度高时，损失接近 0；当模型以很高置信度给出错误预测时，损失会迅速增大。

训练集上的代价函数

对 m 个训练样本取平均，逻辑回归的代价函数为

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h_{\theta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{x}^{(i)}))].$$

训练目标是求

$$\underset{\theta}{\text{minimize}} J(\theta).$$

这个损失也称为**二元交叉熵 (Binary Cross-Entropy)**。

2.2 梯度下降

逻辑回归的梯度

对每个参数 θ_j ，代价函数的偏导数为

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}, \quad j = 0, 1, \dots, n.$$

梯度下降同时更新所有参数：

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}.$$

它在形式上与线性回归的更新式相似，但两者的假设函数不同：线性回归使用 $\boldsymbol{\theta}^T \mathbf{x}$ ，逻辑回归使用 $g(\boldsymbol{\theta}^T \mathbf{x})$ 。

向量化与特征缩放

令特征矩阵 $X \in \mathbb{R}^{m \times (n+1)}$ 、标签向量 $\mathbf{y} \in \mathbb{R}^m$ ，全部样本的预测向量为

$$\hat{\mathbf{y}} = g(X\boldsymbol{\theta}),$$

其中 g 对向量中的每个元素分别计算。于是

$$\nabla J(\boldsymbol{\theta}) = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y}).$$

逻辑回归同样受特征尺度影响。训练前进行特征缩放，通常能使梯度下降更快收敛。

实现交叉熵时的数值问题

由于 $\log(0)$ 没有有限值，直接计算概率形式的交叉熵可能在浮点数下产生无穷大或非数。实践中应使用数值稳定的库函数，或在计算对数前把概率限制在 $(\varepsilon, 1 - \varepsilon)$ 内。该处理只用于避免数值溢出，不应改变损失函数的含义。

2.3 高级优化算法

梯度下降以外的选择

共轭梯度、BFGS 和 L-BFGS 等算法也可以最小化逻辑回归的代价函数。使用这类优化器时，通常需要提供能够返回 $J(\theta)$ 与 $\nabla J(\theta)$ 的函数。它们常常收敛更快，而且不需要手工指定固定学习率，但内部实现更复杂，实际编程时应优先调用经过验证的数值优化库。

3 多类别分类

一对多方法

当 $y \in \{1, 2, \dots, K\}$ 时，可以把一个 K 类问题拆成 K 个二元分类问题。第 k 个分类器把“类别 k ”作为正类，把其他所有类别作为负类，得到

$$h_{\theta^{(k)}}(\mathbf{x}) = P(y = k \mid \mathbf{x}).$$

预测时选择输出最大的分类器：

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} h_{\theta^{(k)}}(\mathbf{x}).$$

这种方法称为**一对多 (One-vs-Rest)** 或一对其余。

三类别分类

若任务需要识别三种类别，则分别训练三个逻辑回归分类器：

- 分类器 1：类别 1 对类别 2、3；
- 分类器 2：类别 2 对类别 1、3；
- 分类器 3：类别 3 对类别 1、2。

对新样本同时计算三个输出。例如输出分别为 0.10、0.72 和 0.18，则预测为类别 2。

4 过拟合与正则化

4.1 欠拟合与过拟合

模型拟合的三种状态

- **欠拟合 (Underfitting)**: 模型过于简单, 连训练数据的主要规律也没有学到, 通常表现为高偏差;
- **合适拟合**: 模型抓住主要规律, 并能较好地推广到未见数据;
- **过拟合 (Overfitting)**: 模型对训练数据拟合得非常好, 甚至代价接近 0, 却把噪声也当成规律, 因而在新数据上表现较差, 通常表现为高方差。

特征很多、样本较少或多项式次数过高时, 过拟合风险尤其明显。

处理过拟合的基本思路

常见方法有两类:

1. 减少特征数量, 人工选择或通过模型选择方法保留真正有用的特征;
2. 使用正则化, 保留特征但限制参数的大小, 使模型更平滑、更简单。

删除特征可能损失信息; 当多数特征都有一定作用时, 正则化通常更合适。

4.2 正则化代价函数

L_2 正则化

在原代价函数后增加参数平方和作为惩罚项:

$$J_{\text{reg}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2.$$

其中 $\lambda \geq 0$ 是**正则化参数 (Regularization Parameter)**。按照惯例, 偏置参数 θ_0 不参与正则化, 因此求和从 $j = 1$ 开始。

正则化参数 λ 的作用

正则化需要在拟合训练数据和限制模型复杂度之间取得平衡:

- $\lambda = 0$ 时没有正则化, 复杂模型可能过拟合;
- λ 合理时, 较大的参数受到抑制, 模型通常具有更好的泛化能力;
- λ 过大时, 除偏置外的参数都被压到接近 0, 模型过于简单并产生欠拟合。

λ 不应根据测试集表现反复调整, 而应使用验证集等模型选择方法确定。

4.3 正则化线性回归

代价函数与梯度下降

正则化线性回归的代价函数为

$$J(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2.$$

偏置和其余参数的更新规则分别为

$$\begin{aligned}\theta_0 &\leftarrow \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_0^{(i)}, \\ \theta_j &\leftarrow \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right], \quad j \geq 1.\end{aligned}$$

第二个式子也可写成

$$\theta_j \leftarrow \left(1 - \alpha \frac{\lambda}{m}\right) \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}.$$

这说明每次更新都会先把参数缩小一点，再沿原代价函数的负梯度方向移动。

正则化正规方程

若使用正规方程，正则化线性回归的解为

$$\boldsymbol{\theta} = (X^T X + \lambda L)^{-1} X^T \mathbf{y}, \quad L = \text{diag}(0, 1, \dots, 1).$$

矩阵 L 左上角为 0，保证 θ_0 不被正则化。实际实现中应使用线性方程求解器或伪逆，而不是显式计算矩阵逆。

4.4 正则化逻辑回归

正则化逻辑回归的代价函数

在二元交叉熵后加入同样的平方惩罚项：

$$\begin{aligned}J(\boldsymbol{\theta}) = & -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))] \\ & + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2.\end{aligned}$$

其梯度为

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_0^{(i)},$$
$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j, \quad j = 1, \dots, n.$$

正则化中的常见错误

- 把 θ_0 加入惩罚项，导致整体预测基线也被不必要地压缩；
- 在代价函数中使用 $\lambda/(2m)$ ，却在梯度中遗漏正则化项的导数 $\lambda\theta_j/m$ ；
- 看到线性回归与逻辑回归的梯度公式外形相似，就误以为二者相同；它们使用的 $h_{\theta}(\mathbf{x})$ 不同；
- 认为 λ 越大越好。过强的正则化会让模型从过拟合转为欠拟合；
- 只比较训练误差来选择模型复杂度和 λ ，从而无法判断模型对新数据的泛化能力。

5 本周知识脉络

从概率模型到泛化能力

第三周的核心流程可以概括为：

1. 用 $\theta^T \mathbf{x}$ 汇总输入特征；
2. 用 Sigmoid 函数把结果转成正类概率；
3. 用阈值和判定边界得到类别预测；
4. 用二元交叉熵衡量概率预测与真实标签的差异；
5. 用梯度下降或高级优化算法最小化代价函数；
6. 对多类别任务训练多个一对多分类器；
7. 当模型过于复杂时，用正则化限制参数大小，并通过合适的 λ 平衡欠拟合与过拟合。

逻辑回归解决的是“如何分类”，正则化进一步解决的是“如何让分类器在未见数据上仍然有效”。